

Intelligent Candidate Discovery & Ranking

Ranking 100,000 candidates the way a great recruiter would — beyond keyword filters



The Problem: Keyword Matching Lies

The Trap Built Into This Dataset

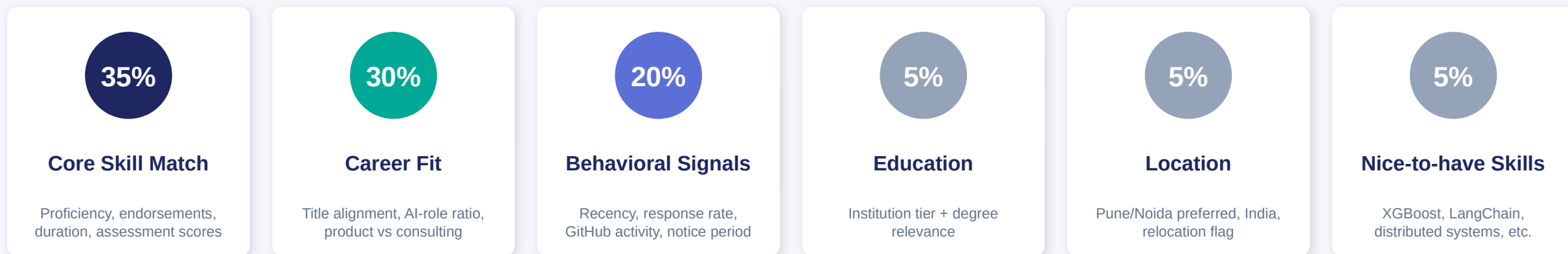
- A candidate titled "Marketing Manager" lists 9 AI keywords as skills
- Zero production AI experience in their career history
- Keyword-based ATS systems rank them in the top 10
- A real ML engineer who never wrote "RAG" or "Pinecone" gets buried

What the JD Actually Asks For

- Understand the GAP between what's said and what's meant
- Career history shows real production ML/ranking/retrieval work
- Down-weight perfect-on-paper but inactive candidates
- Penalize consulting-only careers & job-hoppers

Our Approach: Multi-Signal Weighted Ranking

No API calls. No GPU. Pure Python on CPU. ~15 seconds for 100,000 candidates.



– Keyword Stuffing Penalty

Many listed skills but low average endorsements per skill → penalty applied to final score

Final Score = $(0.35 \times \text{Skills}) + (0.30 \times \text{Career}) + (0.20 \times \text{Behavioral}) + (0.05 \times \text{Education}) + (0.05 \times \text{Location}) + (0.05 \times \text{Nice-to-have}) - \text{Stuffing Penalty}$

Skill Scoring: Trust, Don't Just Count

Each skill is weighted by signals that indicate REAL expertise, not just a listed word

Signal	Weight	Why it matters
Proficiency level	50%	Advanced > Intermediate > Beginner — self-reported but a real signal
Endorsements	30%	Capped at 50; high endorsements = peer validation of real skill
Duration of use	20%	Capped at 3 years; longer use = deeper familiarity
Skill assessment score	+20% bonus	Redrob's own tested score — strongest trust signal available

Keyword Stuffing Detector

- If a candidate lists 10+ skills AND their average endorsements per skill is low →
- $\text{penalty} = \max(0, 0.3 - (\text{avg_endorsements}/50) \times 0.3)$ subtracted from final score

Career Fit: Reading Between the Lines

Title Alignment (35%)

- 1.0 — ML/AI Engineer, Data Scientist, NLP/Search/Ranking Engineer
- 0.6 — Generic "Engineer" or "Scientist" titles
- 0.05 — Marketing/HR/Accountant/Sales/Ops Manager etc. (JD disqualifiers)

AI Experience Ratio (30%)

- % of total career months spent in AI/ML-relevant roles
- Detected via title keywords AND job description text
- 67%+ of career in AI roles → full score

Product vs Consulting (20%)

- % of career at non-consulting companies
- TCS/Infosys/Wipro/Accenture/Cognizant/Capgemini/HCL flagged
- Entirely consulting-only career → -0.4 penalty (per JD)

Years of Experience (15%)

- 5-9 yrs → 1.0 (JD's stated sweet spot)
- 4-5 yrs → 0.8 | 9-12 yrs → 0.7
- Job-hopping (avg tenure < 12mo) → up to -0.3 penalty

Behavioral Signals: Is This Person Even Available?

"A perfect-on-paper candidate who hasn't logged in for 6 months and has 5% recruiter response rate is, for hiring purposes, not actually available." — JD

25%

Recency

≤30 days = 1.0, >365 days = 0.1

20%

Open to Work

Flag true = 1.0, false = 0.3

20%

Response Rate

Direct recruiter response rate value

10%

Profile Completeness

Redrob's completeness score / 100

10%

GitHub Activity

Strong signal for engineers specifically

10%

Notice Period

≤30 days = 1.0 (JD prefers fast joiners)

5%

Verification

Email + phone verified

Results: Top Candidates from 100,000

Rank	Candidate ID	Score	Profile Summary
1	CAND_0002025	0.6916	Senior AI Engineer, 5.9 yrs — strong core skill + career fit
2	CAND_0086022	0.6852	Senior Applied Scientist, 5.3 yrs — highest skill match (0.76)
3	CAND_0046064	0.6848	Senior NLP Engineer, 8.9 yrs — within ideal experience band
4	CAND_0048558	0.6765	Data Scientist, 6.7 yrs — strong career & behavioral signals
5	CAND_0071974	0.6708	Senior AI Engineer, 7.8 yrs — well-rounded profile

0.69

Max Score

0.36

P90 Score

0.27

Median Score

~15 sec

Runtime (100K)

Score distribution shows a clear, meaningful gradient — not a flat distribution of false positives. The system found a narrow band of strong matches, exactly as the JD expects ("10 great matches, not 1000 maybes").

Built for the Constraints That Matter

1

No network during ranking

Zero API calls. Fully offline scoring.

2

No GPU required

Pure Python + standard library logic.

3

Fast & reproducible

~15 seconds for 100K candidates, deterministic output.

4

Explainable by design

Every candidate gets a human-readable reasoning string.

Reproduce in one command

```
pip install -r requirements.txt

python rank.py \
  --candidates ./data/candidates.jsonl \
  --out ./submission.csv
```

Output: submission.csv
candidate_id, rank, score, reasoning

Files: rank.py · requirements.txt · submission_metadata.yaml ·
README.md